

BÁO CÁO BÀI TẬP LỚN — NHÓM 12

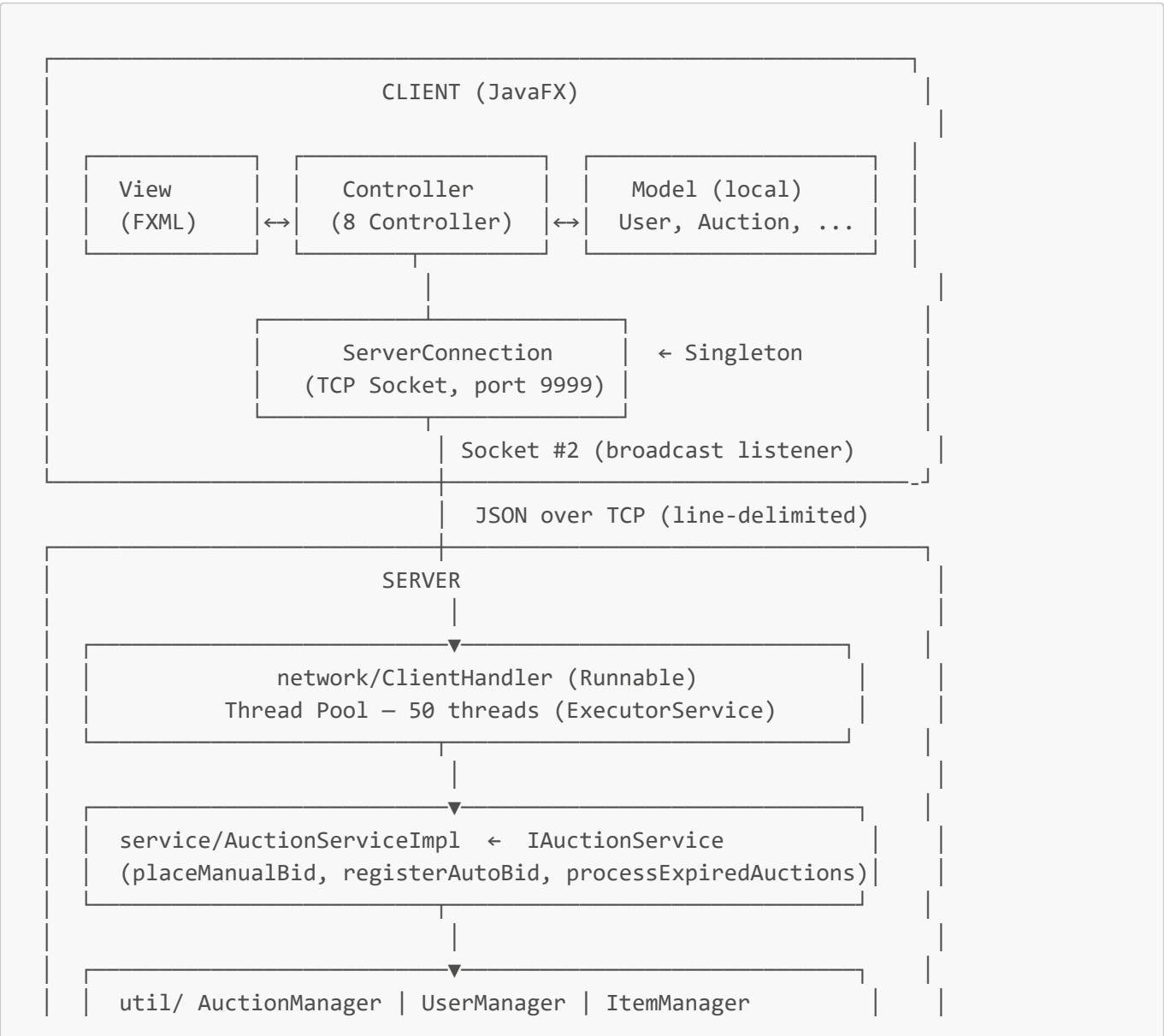
Hệ thống Đấu Giá Trực Tuyến (Online Auction System)

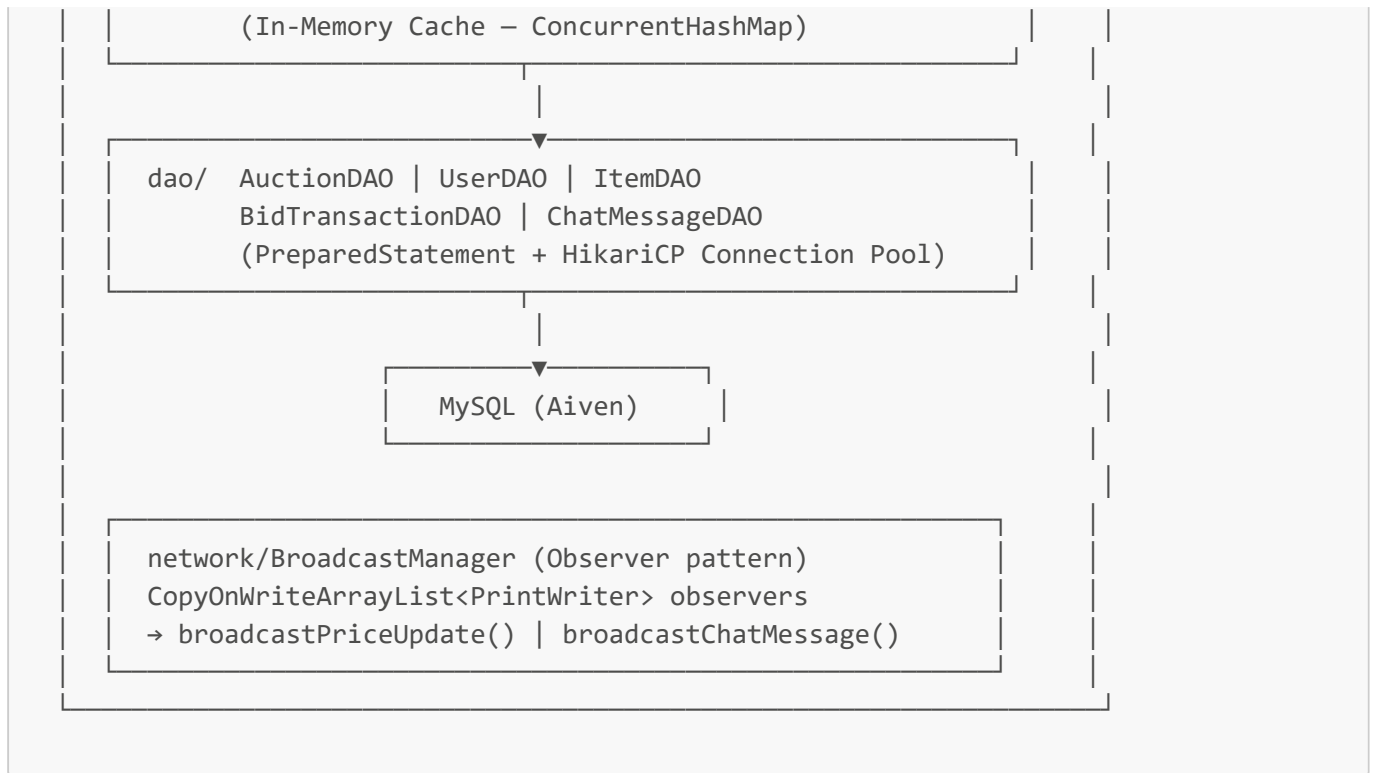
1. GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI

Xây dựng ứng dụng đấu giá trực tuyến theo mô hình **Client-Server**: nhiều người dùng đăng ký tài khoản, đăng bán sản phẩm, đặt giá theo thời gian thực và theo dõi kết quả. Hệ thống áp dụng đầy đủ **OOP, Design Patterns**, xử lý **đồng thời** và kiến trúc phần mềm chuẩn mực.

Phạm vi: Server TCP xử lý nghiệp vụ đấu giá, đồng bộ MySQL, phát sóng realtime. Client JavaFX: đăng ký/ đăng nhập, duyệt/tìm kiếm sản phẩm, đặt giá thủ công và tự động, biểu đồ giá, live chat. Tính năng tùy chọn: Auto-Bidding, Anti-sniping, Bid History Visualization. Hạ tầng: Maven multi-module, JUnit 5, GitHub Actions CI/CD, MySQL (Aiven Cloud) + HikariCP.

2. KIẾN TRÚC TỔNG THỂ HỆ THỐNG





Hệ thống chia làm hai module Maven độc lập (`auction_server`, `auction_client`), giao tiếp qua **TCP Socket** với giao thức **JSON line-by-line**. Server tổ chức 4 tầng: `network` → `service` → `util/cache` → `dao`. Khi khởi động, dữ liệu MySQL nạp lên RAM (`ConcurrentHashMap`) — thao tác đặt giá chỉ cần đọc/ghi RAM, tốc độ cao. Mỗi client được xử lý bởi một `ClientHandler` trong thread pool 50 luồng. Cập nhật realtime qua `BroadcastManager` (Observer). Client theo **MVC**: FXML (View) + 8 Controller + `ServerConnection` Singleton; UI cập nhật qua `Platform.runLater()`.

3. CÁC CHỨC NĂNG ĐẠT ĐƯỢC

3.1 Thiết kế lớp và cây kế thừa (0.5đ)

```

Entity (abstract – int id)
├── User (abstract – username, email, passwordHash, phone, displayName)
│   ├── Bidder (autoBidEnabled, maxAutoBidAmount, autoBidIncrement)
│   ├── Seller (rating)
│   └── Admin
├── Item (abstract – name, description, startingPrice, imagePath)
│   ├── Electronics | Arts | Fashion | RealEstate | OtherItem
└── Auction, BidTransaction (extend Entity)

```

`Entity` là lớp gốc chứa `id`; `User` và `Item` là abstract buộc lớp con override `getCategory()`. Thiết kế này đảm bảo tính mở rộng — thêm loại sản phẩm hoặc vai trò mới mà không sửa code hiện có.

3.2 Áp dụng OOP đầy đủ (1.0đ)

- **Encapsulation:** Mọi field `private`, getter/setter; `currentHighestBid` chỉ sửa trong `synchronized(auction)`.
- **Inheritance:** Cây kế thừa 3 tầng; server và client có package model riêng biệt, tách biệt deployment.

- **Polymorphism:** `IAuctionService` → `AuctionServiceImpl`; `ItemFactory.createItem()` trả về `Item` nhưng thực là subclass cụ thể; `getCategory()` override ở từng loại sản phẩm.
- **Abstraction:** `User`, `Item` là abstract — không khởi tạo trực tiếp; `IAuctionService` định nghĩa hợp đồng nghiệp vụ.

3.3 Design Patterns (1.0đ)

Pattern	Vị trí áp dụng
Singleton	<code>ServerConnection</code> , <code>UserSession</code> , <code>UserManager</code> , <code>AuctionManager</code> , <code>ItemManager</code> (Double-checked locking / Init-on-demand holder)
Observer	<code>BroadcastManager</code> + kênh <code>SUBSCRIBE_PRICE</code> — server push giá và chat đến tất cả client
Factory Method	<code>ItemFactory.createItem(category, ...)</code> — tạo đúng subclass <code>Item</code> theo category string
DAO	5 DAO class — tách hoàn toàn logic DB khỏi nghiệp vụ, PreparedStatement chống SQL injection
Strategy	<code>IAuctionService</code> interface + <code>AuctionServiceImpl</code> — để mock khi Unit Test

3.4 Quản lý người dùng và sản phẩm (1.0đ)

Người dùng: Đăng ký với BCrypt hash (cost 12), phân quyền Bidder/Seller/Admin. Đăng nhập qua BCrypt verify. Cập nhật email/SDT. Admin xem/xóa user (cascade: auction → bids → chat → user).

Sản phẩm: `ADD_ITEM` → `ItemFactory` tạo đúng loại → lưu DB → tự động tạo phiên `Auction` RUNNING. Gallery nhiều ảnh (Cloudinary URL). Quản lý tab: đang bán / đang đấu giá / đã bán / đã tham gia. Admin xóa sản phẩm (cascade).

3.5 Chức năng đấu giá (1.0đ)

Luồng: `PLACE_BID` → kiểm tra hợp lệ → `synchronized(auction)` → cập nhật RAM + DB → broadcast giá mới đến tất cả client.

Trạng thái phiên: `RUNNING` → `PENDING_PAYMENT` (10 phút) → `FINISHED` / `CANCELLED`.

`ScheduledExecutorService` kiểm tra mỗi 30 giây, tách biệt với luồng xử lý bid để tránh bottleneck. Người thắng xác nhận/từ chối thanh toán trong 10 phút, quá hạn tự động `CANCELLED`.

3.6 Xử lý lỗi và ngoại lệ (1.0đ)

`AuctionException` (cha) → `BidTooLowException` (con): phân biệt lỗi nghiệp vụ cụ thể. Xử lý phân tầng: service ném exception → `ClientHandler` bắt và trả message rõ ràng → `ServerConnection` client xử lý timeout 10s / `IOException` với auto-reconnect → Controller hiển thị lỗi thân thiện (`Alert`, label màu đỏ).

3.7 Xử lý đồng thời — Concurrency (1.0đ)

- `synchronized(auction)`: lock per-object, nhiều phiên vẫn chạy song song — tránh bottleneck toàn hệ thống.
- `ConcurrentHashMap`: lưu trữ auction/user thread-safe trong `AuctionManager`, `UserManager`.
- `CopyOnWriteArrayList`: `BroadcastManager` add/remove observer an toàn khi nhiều luồng đồng thời.

- Thread pool 50 luồng + `ExecutorService (newCachedThreadPool)`: Auto-Bid chạy bất đồng bộ, không block luồng bid chính.
- Trường `version` trên `Auction`: optimistic concurrency, reconcile dữ liệu khi restart server.

3.8 Realtime Update — Observer/Socket (0.5đ)

Client mở socket thứ hai, gửi `SUBSCRIBE_PRICE`. Server lưu `PrintWriter` vào `BroadcastManager`. Khi có bid/chat mới → `broadcastPriceUpdate()` / `broadcastChatMessage()` đẩy JSON ngay đến mọi observer. Client nhận trên background thread → `Platform.runLater()` cập nhật UI. Client ngắt kết nối → tự xóa khỏi danh sách. Tách kênh broadcast khỏi request-response để tránh xung đột protocol.

3.9 Kiến trúc Client–Server (0.5đ)

Server port 9999, JSON/TCP line-by-line, thread pool 50, client/server khác package và khác JAR. `ServerConnection` Singleton quản lý một kết nối TCP duy nhất, auto-reconnect, timeout 10s. (Chi tiết đã trình bày ở Mục 2.)

3.10 MVC — JavaFX + FXML (0.5đ)

8 cặp View/Controller: `login`, `signup`, `home`, `selling_product`, `add_product`, `my_products`, `profile`, `admin`. `Main.changeScene()` swap root Scene để điều hướng. Controller chỉ xử lý sự kiện UI, không chứa logic DB.

3.11 Maven & Coding Convention (0.5đ)

Multi-module Maven (`auction_server/pom.xml`, `auction_client/pom.xml`), build `mvn clean verify`. Naming convention Java chuẩn (camelCase/PascalCase). Comment logic phức tạp. Hằng số đặt tên rõ ràng (`TIMEOUT_MS = 10_000`).

3.12 Unit Test — JUnit 5 (0.5đ)

`AuctionServiceImplTest.java` — 3 test case: (1) `assertThrows(BidTooLowException.class)` khi đặt giá thấp; (2) `assertEquals` giá và người thắng sau bid hợp lệ; (3) `assertTrue(endTime.isAfter(...))` xác nhận anti-sniping gia hạn thời gian khi bid trong 10 giây cuối.

3.13 CI/CD — GitHub Actions (0.5đ)

Trigger push/PR vào `main`, `master`, `develop`, `dev`. Job `build-and-test`: JDK 17 Temurin, `mvn -B clean verify`, upload JAR (14 ngày). Job `deploy` (chỉ `main/master`): đóng gói release bundle (30 ngày). `cancel-in-progress: true` hủy run cũ khi có run mới.

3.14 Auto-Bidding (0.5đ — Tùy chọn)

Người dùng nhập `maxAmount` + `increment`. Server thêm `Bidder` vào `autoBidders`. Sau bid thật → `notifyAutoBidObservers()` → submit `ExecutorService` → bot ngủ 1.5s → `processBid(isAutoBid=true)`. Cờ `isAutoBid` chặn đệ quy vô hạn giữa các bot.

3.15 Anti-Sniping (0.5đ — Tùy chọn)

Trong `processBid()`: bid trong **30 giây cuối** → tự động gia hạn **+60 giây**. Chạy trong `synchronized(auction)`, thread-safe và minh bạch với người dùng.

3.16 Bid History Visualization (0.5đ — Tùy chọn)

Danh sách lịch sử đặt giá + `LineChart` diễn biến giá, cập nhật realtime qua Observer nội bộ (`dialogHistoryBox`, `dialogChartSeries`). Auto-scale trục, dark theme CSS, label giá trên từng điểm dữ liệu.

4. PHÂN CHIA CÔNG VIỆC

STT	Thành viên	Nhiệm vụ chính	Barem
1	Trọng Tùng	Cây kế thừa & 4 nguyên lý OOP; tầng DAO (5 class) + HikariCP; Maven + coding convention	0.5 + 1.0 + 0.5
2	Minh Sơn	<code>AuctionServiceImpl</code> + concurrency (<code>synchronized</code> , <code>ConcurrentHashMap</code>); Anti-sniping; xử lý lỗi/ngoại lệ; scheduler & thanh toán	1.0 + 0.5 + 1.0
3	Tuấn Minh	<code>ClientHandler</code> + network server (thread pool, shutdown hook); <code>BroadcastManager</code> Observer; Design Patterns; Auto-Bidding	0.5 + 0.5 + 1.0 + 0.5
4	Hải Nam	JavaFX/FXML 8 màn hình; Bid History Visualization; quản lý user/sản phẩm UI; JUnit 5 (3 test); CI/CD GitHub Actions	0.5 + 0.5 + 1.0 + 0.5 + 0.5

Mỗi thành viên hỗ trợ nhau trong tích hợp và debug. Bảng phản ánh người chịu trách nhiệm chính.